COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	8/18//2026	**DATE SUBMITTED:**	8/27//2026
NAME:	Denzen D. Igloso	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. _6.1__ Searching Techniques

1. PROCEDURE OUTPUT

Screenshot

```
8   const int max_size = 50;
9   int main() {
10      //generate random values:
11      int dataset[max_size];
12      srand(time(0));
13      for(int i=0; i<max_size; i++){
14          dataset[i] = rand();
15      }
16      for(int i=0; i < max_size; i++){
17          std :: cout << dataset[i] << " ";
18      }
19      std::cout << std::endl;
```

Output

```
23585 11644 22627 1846 1751 21395 31413 19648 29181 6939 21614 19675 28433 15337 31013 23449 5493 29993 23395 20779 3360 32068 3321 6380 29076 10504 22934 12705 2
4848 3587 28568 766 3078 9263 2851 11373 22735 10965 22384 12556 28574 21675 31336 9460 17075 32575 32370 16321 26321 31129
what value you want to search?
```

Observation	Using srand(time(0)) seeds the random generator so new numbers appear on every run. The loop then populates all 50 indices of the array with random values.

Table 6-1. Data Generated and Observations.

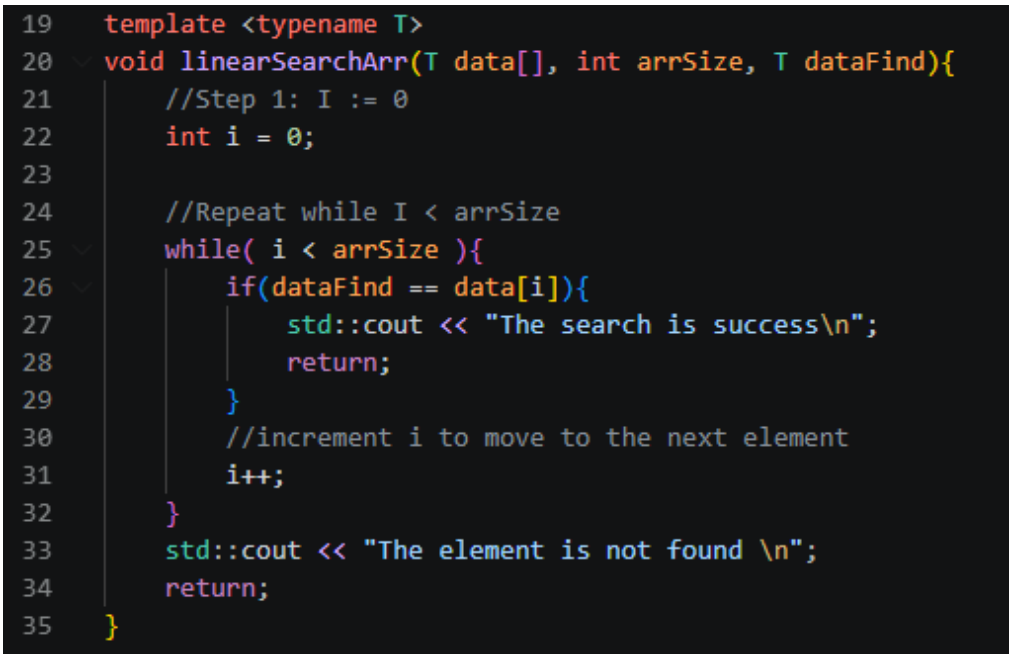
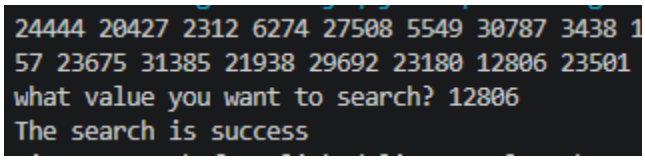
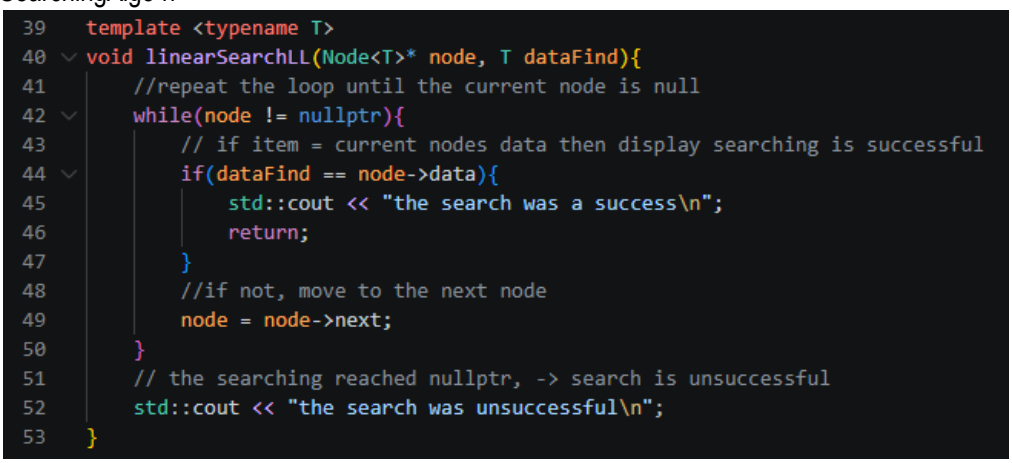
Code	
Output	
Observation	Linear search checks array elements sequentially from start to end. It stops and outputs a result as soon as a matching value is found or the array ends.

Table 6-2a. Linear Search for Arrays

Code	SearchingAlgo h  Main cpp

	47 std::cout << "Linear search for linked list result: "; 48 linearSearchLL(name1, 'D'); 49 std::cout << "Linear search for linked list result: "; 50 linearSearchLL(name1, 'x');
Output	24444 20427 2312 6274 27508 5549 30787 3438 13585 28397 15483 31260 182 57 23675 31385 21938 29692 23180 12806 23501 6341 4642 14249 27369 7068 what value you want to search? 12806 The search is success Linear search for linked list result: the search was a success
Observation	Searching the linked list follows the same logic, but moves node-by-node using node = node->next. Reaching nullptr means the target item wasn't in the list.

Table 6-2b. Linear Search for Linked List

Code	SearchingAlgo h
	98 template <typename T>
	99 void binarySearchArr(T data[], int arrSize, T findData) {
	100 int low = 0;
	101 int high = arrSize - 1;
	102
	103 while (low <= high) {
	104 int mid = low + (high - low) / 2;
	105
	106 if (data[mid] == findData) {
	107 std::cout << "The search was a success\n";
	108 return;
	109 }
	110 if (data[mid] < findData) {
	111 low = mid + 1;
	112 } else {
	113 high = mid - 1;
	114 }
	115 }
	116
	117 std::cout << "The search was unsuccessful\n";
	118 }
	119
	120 #endif
	Main cpp
	56 std::cout << "Binary search for array result: ";
	57 binarySearchArr(dataset, max_size, toFind);
	58 }

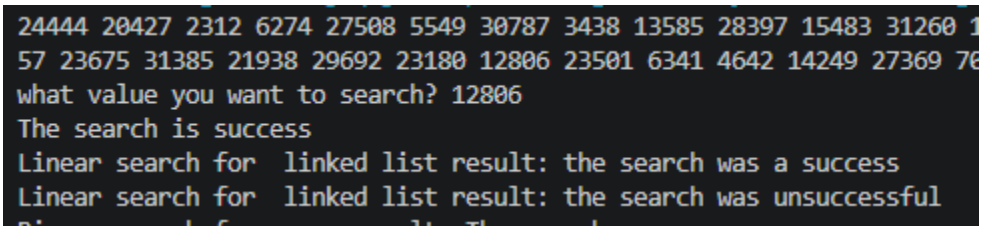
Output	
Observation	Binary search cuts the search range in half on each step, making it much faster. However, std::sort() must run first because binary search only works on sorted data.

Table 6-3a. Binary Search for Arrays

Code	searchingAlgo h
	``
	template <typename T>
	Node<T>* getMiddle(Node<T>* start, Node<T>* last){
	    //use two pointers: 1. slow one node at a time, fast moves two at a time
	    Node<T>* slow = start;
	    Node<T>* fast = start;
	
	    //get middle:
	    while(fast != last && fast->next != last){
	        //move slow (one step)
	        slow = slow->next;
	        //move fast (two steps)
	        fast = fast->next->next;
	    }
	    //when fast reaches the end
	    // slow points to the middle node
	    return slow;
	}
	``

```cpp
template <typename T>
void binarySearchLL(Node<T>* start, T findData) {
    Node<T>* last = nullptr;

    while (start != last) {
        Node<T>* mid = getMiddle(start, last);

        if (mid == nullptr) {
            std::cout << "The search was unsuccessful\n";
            return;
        }

        if (mid->data == findData) {
            std::cout << "The search was a success\n";
            return;
        } else if (mid->data < findData) {
            start = mid->next;
        } else {
            last = mid;
        }
    }

    std::cout << "The search was unsuccessful\n";
}
```

Output

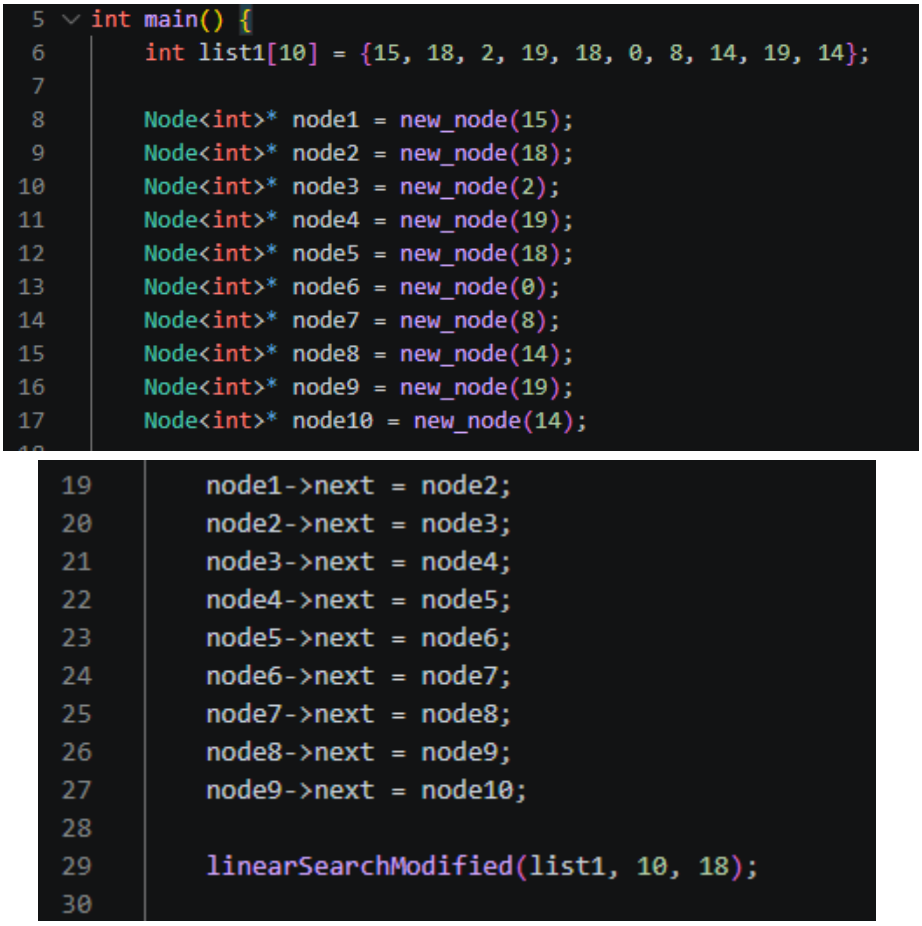
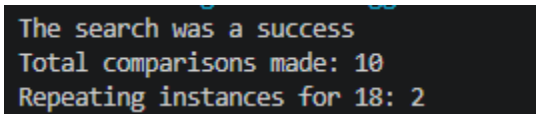
```
24444 20427 2312 6274 27508 5549 30787 3438 13585 28397 15483 31260 18
57 23675 31385 21938 29692 23180 12806 23501 6341 4642 14249 27369 706
what value you want to search? 12806
The search is success
Linear search for  linked list result: the search was a success
Linear search for  linked list result: the search was unsuccessful
Binary search for array result: The search was a success
```

Observation

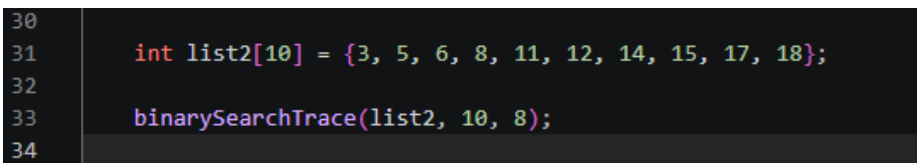
Slow and fast pointers are needed to locate the middle node since linked lists lack direct index access. Moving through pointer links makes this approach slower than array binary search.

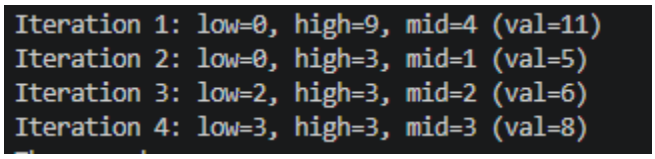
Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

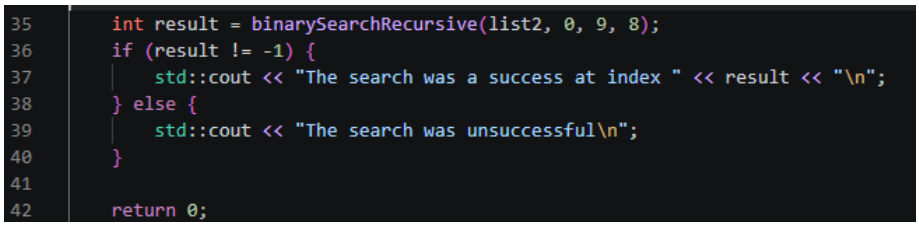
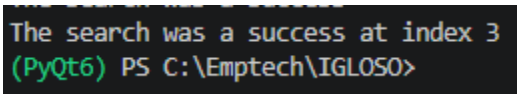
Code	
Output	
Observation	The modified linear search checked every single elements from index 0 to 9, so it made 10 total comparisons just to make sure it doesn't miss any duplicates. It correctly found that the key 18 appears 2 times in the list at index 1 and 4.

Problem 1 and 2

Code	

Output	
Observation	For the binary search trace, it shows how the list is getting cut in half every loop, starting from mid value 11 then narrowing down after 4 iterations. It found key 8 at index 3 which proves that my diagram steps is correct.

Problem 3

Code	
Output	
Observation	The recursive binary search gave the exact same output with the iterative version, finding key 8 at index 3. Instead of using while loop, it just call itself back with the updated low and high values until it finds the key.

Problem 4

Conclusion

- Overall, I think I handled this lab well because I got all the search algorithms working, matched the target outputs, and confirmed my diagram steps using the trace output. My main area for improvement is recursion, specifically training myself to trace the call stack step-by-step mentally rather than relying on cout statements to see the range changes. I also want to practice linked list pointer setups so my code looks cleaner.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google, quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar, and searching for syntax examples.**

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Quilbot was used to check my grammar on observations and google for searching basic syntax to have a references on the things that i dont understand

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.